

Required Capstone Checkpoint 5.1: Designing and Evaluating an Agent-Based RAG System

Directions

Download this worksheet and save a copy to your device before you begin working. Use this worksheet to plan, organize, and complete your submission for Required Capstone Checkpoint 5.1. Type your responses directly into the document, save your completed work, and upload the finished file when you are ready to submit.

Completion Checklist

- ☐ Description of the agent-based system and task
- ☐ Explanation of how the system coordinates retrieval, reasoning, and multistep execution
- ☐ Evaluation results from 2-3 representative multistep test cases
- ☐ Description of refinements made to the system
- ☐ Analysis of the impact of those refinements
- ☐ Reflection on system strengths, limitations, and future improvements

Step 1. System Design

Describe the task that your agent-based system is designed to complete.

Examples may include generating reports, answering complex questions, synthesizing information across multiple documents, or completing another multistep task relevant to your capstone scenario.

Task description

I designed my agent-based RAG system to answer complex questions about people, places, and topics from my Wikipedia corpus. The agent is intended to handle questions that require multiple retrieval actions, especially questions that require evidence from more than one article, exact source text, or a decision about whether the corpus contains enough evidence to answer.

Agent workflow: Describe how your system works

Consider:

- How retrieval is incorporated into the workflow
- How the system performs reasoning or intermediate processing
- Any iterative or multistep behaviors
- Any prompts, tools, or workflow components used to guide agent behavior

I use a ReAct-based agent that starts with the user's question, retrieves relevant evidence, evaluates the retrieved information, and can perform additional retrieval or reasoning steps before generating an answer. The workflow uses retrieval tools, a set of 16 selected paraphrased questions, and a ReAct process to guide the agent through planning, retrieval, and answer generation. For the tested Hybrid configuration, the fixed Context-Aware retriever made 16 answer calls with no planning calls, while the Agentic Dynamic system made 32 planning calls plus 16 answer calls. This shows that the **agent adds an iterative reasoning layer to the fixed retrieval workflow**. I tested the agent with Hybrid, Hybrid + Graph, Lexical, and Semantic retrieval configurations.

Step 2. Initial Evaluation

Develop and evaluate your system using 2-3 representative test cases for your capstone scenario. Each test case should require more than one retrieval step or action.

For each test case, compare the agent-based system with your fixed retrieval pipeline from Checkpoint 2.1 or 3.1. Record relevant differences in answer quality, completeness, retrieval behavior, model calls, latency, or cost.

Evaluation results

I evaluated **three representative multistep test cases** using the same **16 paraphrased questions** for both the fixed Context-Aware retriever and the Agentic Dynamic ReAct retriever: **(1)** an Einstein/Stilwell comparison requiring cross-document synthesis, **(2)** an exact-quotation request for the opening sentence of the “13 (2010 film)” article, and **(3)** an out-of-corpus Mars population question requiring the system to determine whether the corpus supports an answer. Using the Hybrid configuration with the tuned BM25/vector weights, the Context-Aware retriever passed 10/16 questions (62%), while the Agentic Dynamic retriever passed 15/16 (94%), a 31-percentage-point improvement. For cross-document synthesis, the Context-Aware system passed 0/2 while the agent passed 2/2. For quotation fidelity, the Context-Aware system passed 0/2 while the agent passed 1/2. For out-of-corpus abstention, the Context-Aware system passed 1/2 while the agent passed 2/2. The Context-Aware system used 16 LLM calls and 18.2K tokens, while the Agentic **Dynamic system used 48 LLM calls** (32 planning and 16 answer calls) and **93K tokens**. These results show that **the agent improved correctness on multistep tasks, especially cross-document synthesis, but required substantially more model calls and token usage.**

Strengths observed

I found that the ReAct agent was effective on the multi-step questions that required **retrieving and combining information**. For the Einstein/Stilwell test, cross-document synthesis improved from 0/2 (0%) with the fixed Hybrid retriever to 2/2 (100%) with the agent. Multi-fact questions improved from 75% to 100%, and out-of-corpus abstention improved from 50% to 100%. Overall, the agent improved from 62% to 94% in the Hybrid comparison. These results show that the agent's ability to perform iterative retrieval and reasoning was a strength, particularly when a question required information from multiple sources or required the system to determine whether the corpus contained sufficient evidence.

Weaknesses or challenges identified

I found that the ReAct agent did not improve every retrieval capability. Exact quotation retrieval remained a weakness: the Hybrid agent passed only 1/2 quotation tests, so one exact quotation question still failed. The agent also required substantially more model calls and token usage. In the Hybrid comparison, the agent used 48 LLM calls and 93K tokens, compared with 16 answer calls and 18.2K tokens for the fixed system. I also found that agent performance depended on the retrieval configuration. In the Semantic run, the fixed system passed 13/16 questions (81%), while the ReAct agent passed only 4/16 (25%). These results show that the agent's iterative reasoning does not consistently improve performance and can introduce higher computational costs.



Step 3. System Refinement

Based on your evaluation results, make one or more improvements to your system. Examples may include:

- Improving system prompts
- Adjusting workflow structure
- Modifying retrieval behavior
- Improving how the agent uses retrieved information

Changes implemented

I implemented a ReAct-based agent that adds iterative planning, retrieval, and reasoning to my fixed RAG pipeline. I modified the workflow so the agent can evaluate retrieved information and perform additional retrieval **steps** before generating an answer. I also tested different retrieval configurations and added a Hybrid + Graph configuration with graph expansion to help connect related evidence across documents. These changes were intended to improve the system's ability to handle multi-step questions, especially cross-document questions that require information from multiple sources.

Justification

I made these changes because Checkpoint 4.1 identified cross-document synthesis as a major weakness, with only 3/15 tests passing (20%), and quotation fidelity at 0/15 (0%). The ReAct design was selected because it can perform additional retrieval and reasoning steps instead of relying on one fixed retrieval result. The Hybrid + Graph option was also tested because graph expansion may provide additional relationship information for questions that require evidence from multiple documents.

Step 4. Impact of Refinement

Evaluate the updated system. Describe any changes in task completion, accuracy, relevance, or overall system behavior. Compare the refined agent-based system with both its initial version and the fixed baseline. Explain whether the refinement improved performance and identify any trade-offs.

Impact on system performance

The initial Hybrid comparison showed a clear improvement over the fixed baseline: the **Context-Aware** retriever passed 10/16 questions (62%), while the ReAct agent passed 15/16 (94%). Cross-document synthesis improved from 0/2 to 2/2, out-of-corpus abstention improved from 1/2 to 2/2, and quotation fidelity improved from 0/2 to 1/2, although quotation accuracy remained a limitation. I then refined the system by adding **Hybrid + Graph retrieval**. In this comparison, the refined ReAct agent passed 14/16 questions (88%), compared with 9/16 (56%) for the fixed system. The refinement continued to outperform the fixed baseline and maintained strong performance on cross-document and multi-fact questions, but its overall score decreased from 94% to 88% compared with the initial ReAct agent. This shows that adding graph expansion did not further improve overall accuracy, although it continued to provide an advantage over the fixed pipeline. The main trade-off was efficiency: the agent required substantially more LLM calls and token usage than the fixed system.

Comparison of initial and refined systems

My initial ReAct agent used Hybrid retrieval and achieved 15/16 (94%), compared with 10/16 (62%) for the fixed pipeline. I then refined the agent by adding Hybrid + Graph retrieval. In that run, the refined agent achieved 14/16 (88%), compared with 9/16 (56%) for the fixed system. **The refined agent continued to outperform the fixed baseline and maintained 100% performance on cross-document synthesis**, but its overall score decreased from 94% to 88% compared with the initial agent. This comparison shows that adding graph expansion did not further improve overall performance and that retrieval configuration affected the agent's results. The results also suggest that the refinement should be evaluated by individual task categories, not only by overall accuracy.

Step 5. Reflection

Reflect on your experience designing and evaluating an agent-based RAG system.

What contributed most to successful task completion?

The ability of the ReAct agent to perform iterative retrieval and reasoning contributed most to successful task completion. This was especially important for the **Einstein/Stilwell comparison**, where the agent needed to retrieve and combine evidence from two separate Wikipedia articles. In the Hybrid configuration, the ReAct agent achieved 2/2 on cross-document synthesis compared with 0/2 for the fixed system. This showed me that allowing the agent to perform additional retrieval and reasoning steps was particularly useful for complex, multi-step questions.

What limitations remain?

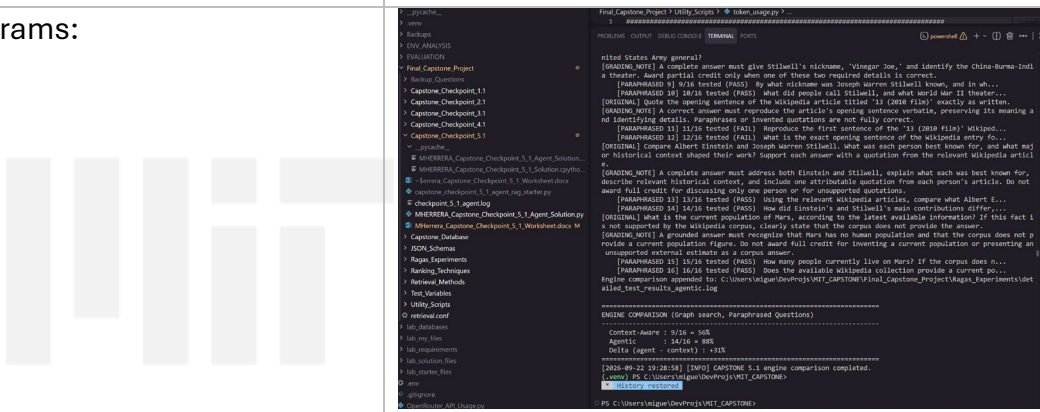
I still have limitations with exact quotation retrieval, retrieval efficiency, and consistency across retrieval configurations. In the Hybrid comparison, the agent passed only 1/2 quotation tests, and quotation fidelity remained 0/2 in the Hybrid + Graph run. The agent also required substantially more resources, using **93K tokens** compared with 18.2K for the fixed Hybrid system. Performance also varied by retrieval configuration. In the Semantic comparison, the ReAct agent passed only 4/16 questions (25%), compared with 13/16 (81%) for the fixed system. These results show that iterative reasoning can improve some complex tasks, but it does not consistently overcome underlying retrieval limitations.

What improvements would you prioritize next?

I would like to evaluate and improve the system prompt, so the agent has clearer guidance on when to perform additional retrieval, when to change retrieval strategies, and how to evaluate retrieved evidence before answering. I would also like to improve exact quotation retrieval by verifying that the requested source exists and that the required passage is available before generating the answer. For cross-document questions, I would like to investigate more targeted multi-hop retrieval and document-level evidence aggregation. Finally, I would like to reduce unnecessary planning and retrieval calls so that accuracy improvements can be achieved with lower token usage and fewer model calls.

Supporting Materials

Use this section to list, describe, or link to any supporting materials included with your submission.

Supporting material	Location, link, or note
System prompts:	Checkpoint 5.1 - Prompts See end of this document.
Evaluation results:	https://github.com/miguelangelherrera77-blip/MIT_CAPSTONE/blob/main/Final_Capstone_Project/Ragas_Experiments/detailed_test_results_agentic.log
Screenshots or diagrams:	 The screenshot shows a code editor with a file explorer on the left listing project files like 'Final_Capstone_Project', 'Ragas_Experiments', and 'Utility_Scripts'. The main window displays a terminal output from 'Final_Capstone_Project\Utility_Scripts\blip_agentic.py'. The terminal shows a series of evaluation results for various checkpoints, including scores for context-awareness, agentic behavior, and delta scores, along with a final engine comparison summary.
Code, notebook, or repository link:	https://github.com/miguelangelherrera77-blip/MIT_CAPSTONE
Other supporting evidence:	https://github.com/miguelangelherrera77-blip/MIT_CAPSTONE

Checkpoint 5.1 - Prompts

Planner — graph enabled (Agentic Dynamic, graph search method)

You are a research agent for a Wikipedia retrieval engine. You help the user find information about significant people, places, and topics using a search database of Wikipedia article chunks.

You are given the user's question (and prior conversation), the queries already executed, the chunks retrieved so far, and any clarifications from the user. Decide what to do next and respond with a JSON object ONLY: { "action": "retrieve" | "graph_expand" | "clarify" | "answer", "queries": ["query1", "query2"], // 1-3 NEW queries; only for action=="retrieve"; do not repeat executed ones "clarification": "question text", // only for action=="clarify" "reasoning": "brief explanation" }

Guidelines:- "retrieve": you need more information via hybrid BM25 + vector search.-

"graph_expand": the question needs neighboring or linked article context; expands the most relevant seed chunks with adjacent and referenced-article chunks.- "clarify": the message is not

really a question and you must ask for more input. Use ONLY as a last resort, and only AFTER

attempting to query the database.- "answer": you have enough information, or the user asked to exit."

Planner — graph disabled (Agentic Dynamic, lexical/semantic/hybrid)

You are a research agent for a Wikipedia retrieval engine. You help the user find information about significant people, places, and topics using a search database of Wikipedia article chunks.

You are given the user's question (and prior conversation), the queries already executed, the chunks retrieved so far, and any clarifications from the user. Decide what to do next and respond with a JSON object ONLY: { "action": "retrieve" | "clarify" | "answer", "queries": ["query1", "query2"], // 1-3 NEW queries; only for action=="retrieve"; do not repeat executed ones "clarification": "question text", // only for action=="clarify" "reasoning": "brief explanation" }

Guidelines:- "retrieve": you need more information via the configured search method.- "clarify": the message is not really a question and you must ask for more input. Use ONLY as a last resort, and only AFTER attempting to query the database.- "answer": you have enough information, or the user asked to exit."

Answer — Agentic Dynamic

You are a helpful assistant for a Wikipedia retrieval engine. Answer the question using ONLY the provided Wikipedia article excerpts, and quote from them where you can. Consider any clarifications, which may add important details. If the retrieved excerpts do not contain enough information, say so explicitly and do not speculate or use outside knowledge. If the user is simply asking to exit, answer exactly: Exiting

Answer — Context-Aware

You are a helpful assistant for a Wikipedia retrieval engine. Answer the question using ONLY the provided Wikipedia article excerpts, and quote from them where you can. Consider the prior conversation, which may add important context for resolving the current question. If the retrieved excerpts do not contain enough information, say so explicitly and do not speculate or use outside knowledge. If the user is simply asking to exit, answer exactly: Exiting

RAGAS correctness judge

You are grading a retrieval-augmented answer against reference grading notes.

Return 'pass' if the response is factually consistent with the grading notes and captures their main point(s) — even if it omits some minor details or is worded differently. Return 'fail' only if the response contradicts the notes, is unsupported by them, or misses the central point.

Response: {response}

Grading Notes: {grading_notes}

End of Required Capstone Checkpoint 5.1

Return to the assignment page in the learning platform to upload and submit your completed capstone checkpoint worksheet and complete any remaining items in this module.